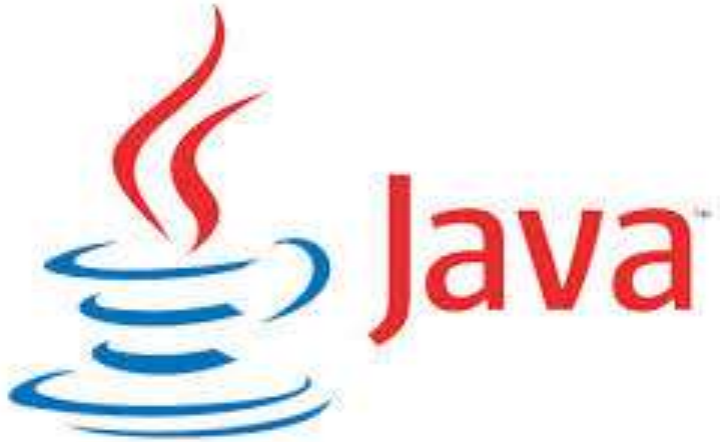


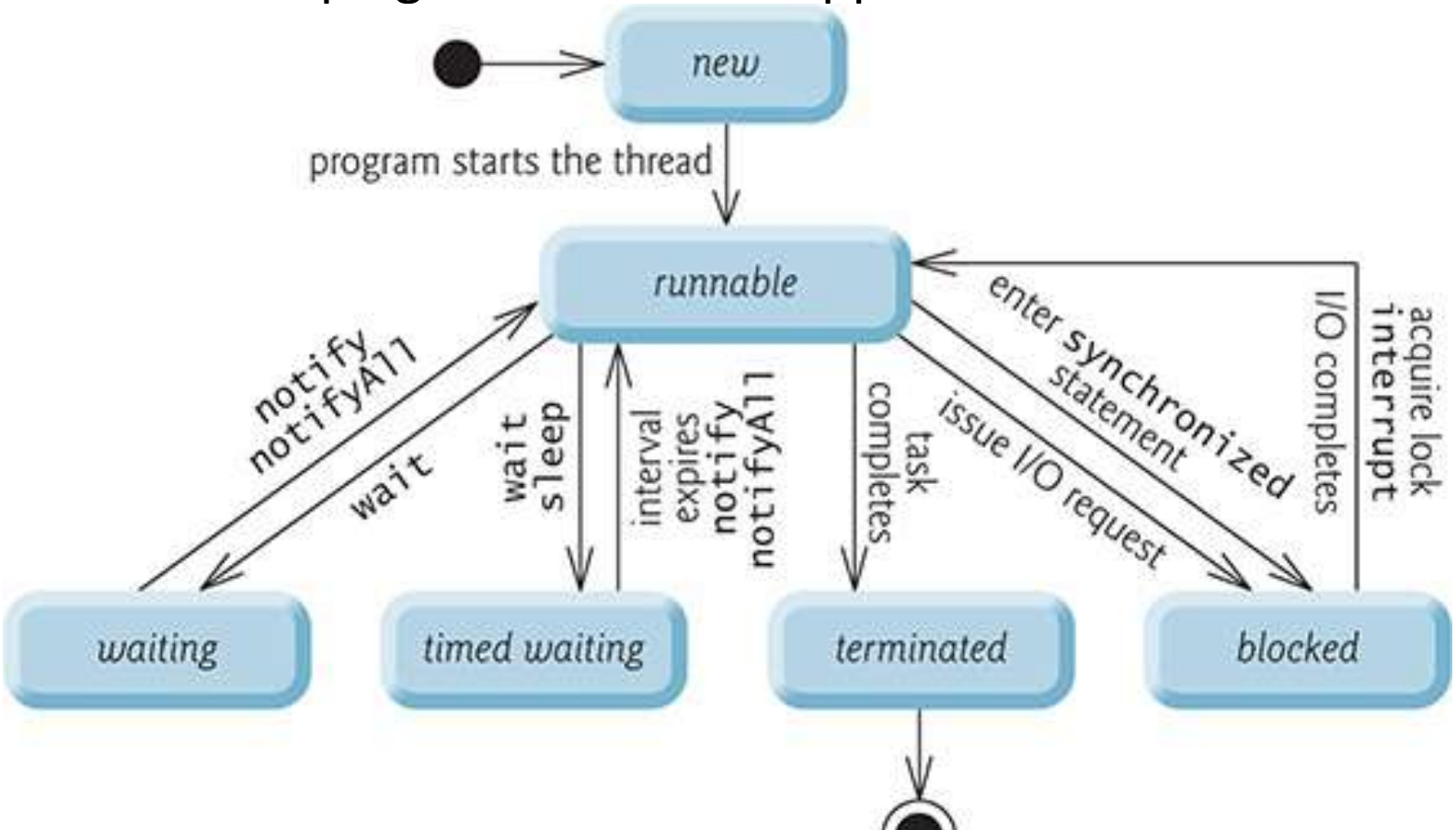
SOC2030 Application Programming in Java

Thread life cycle + synchronization

Dr. Andrei Dragunov

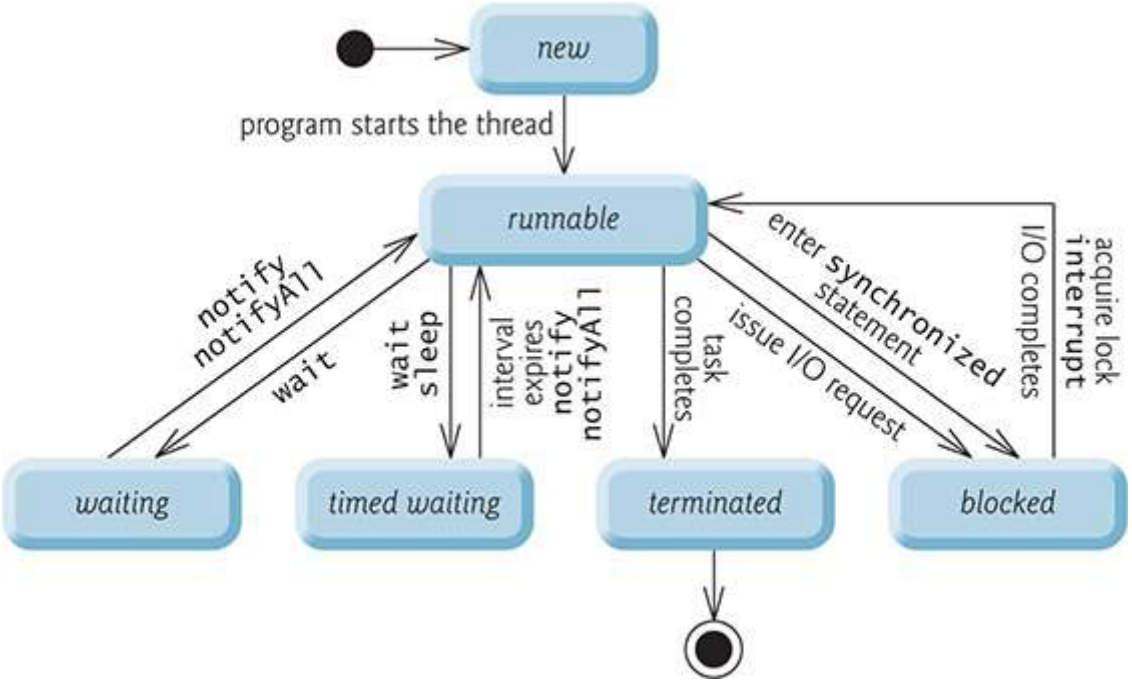
Thread States and Life Cycle

At any time, a thread is said to be in one of several thread states—illustrated in the UML state diagram in Figure Several of the terms in the diagram are defined later. We include this discussion to help you understand what's going on "under the hood" in a Java multithreaded environment. Java hides most of this detail from you, greatly simplifying the task of developing multithreaded applications.



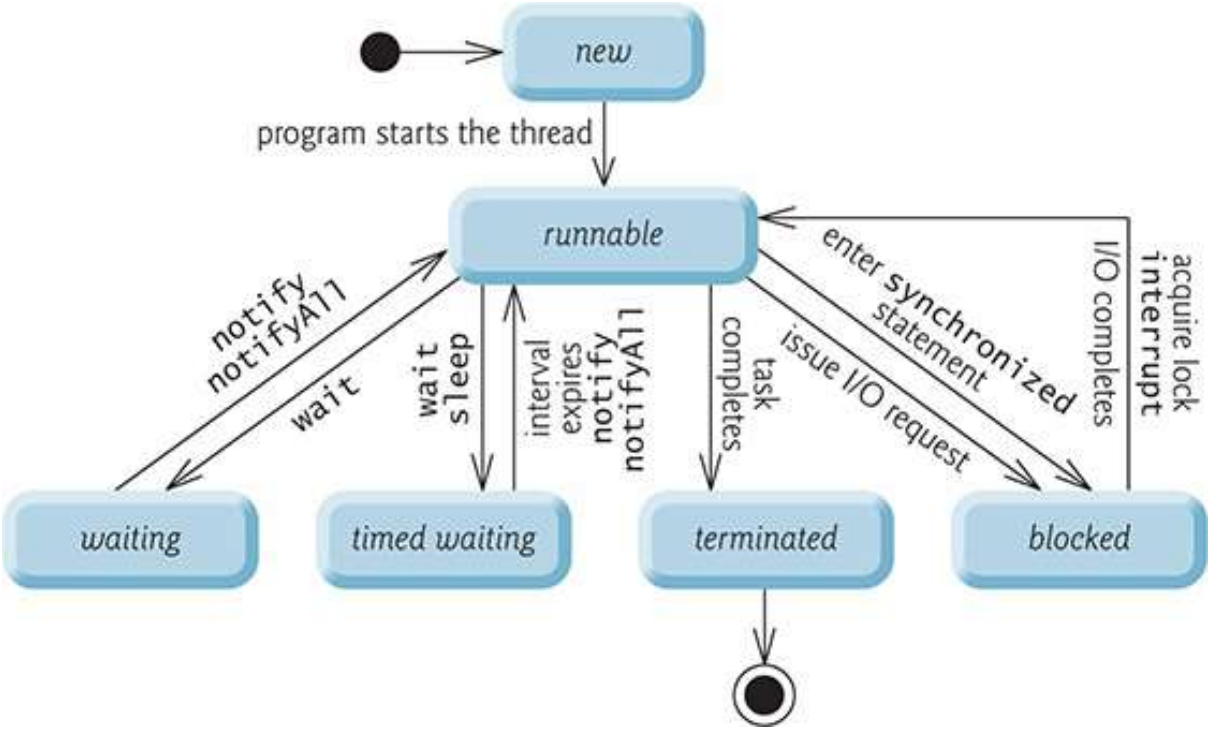
New and *Runnable* States

- A new thread begins its life cycle in the ***new*** state. It remains in this state until the program starts the thread, which places it in the ***runnable*** state. A thread in the *runnable* state is considered to be executing its task.



Waiting State

- Sometimes a *runnable* thread transitions to the ***waiting*** state while it waits for another thread to perform a task. A *waiting* thread transitions back to the *runnable* state only when another thread notifies it to continue executing.



Timed Waiting State

A *runnable* thread can enter the ***timed waiting*** state for a specified interval of time. It transitions back to the *runnable* state when that time interval expires or when the event it's waiting for occurs. *Timed waiting* threads and *waiting* threads cannot use a processor, even if one is available. A *runnable* thread can transition to the *timed waiting* state if it provides an optional wait interval when it's waiting for another thread to perform a task. Such a thread returns to the *runnable* state when it's notified by another thread or when the timed interval expires—whichever comes first. Another way to place a thread in the *timed waiting* state is to put a *runnable* thread to sleep— a **sleeping thread** remains in the *timed waiting* state for a designated period of time (called a **sleep interval**), after which it returns to the *runnable* state. Threads sleep when they momentarily do not have work to perform.

Blocked State

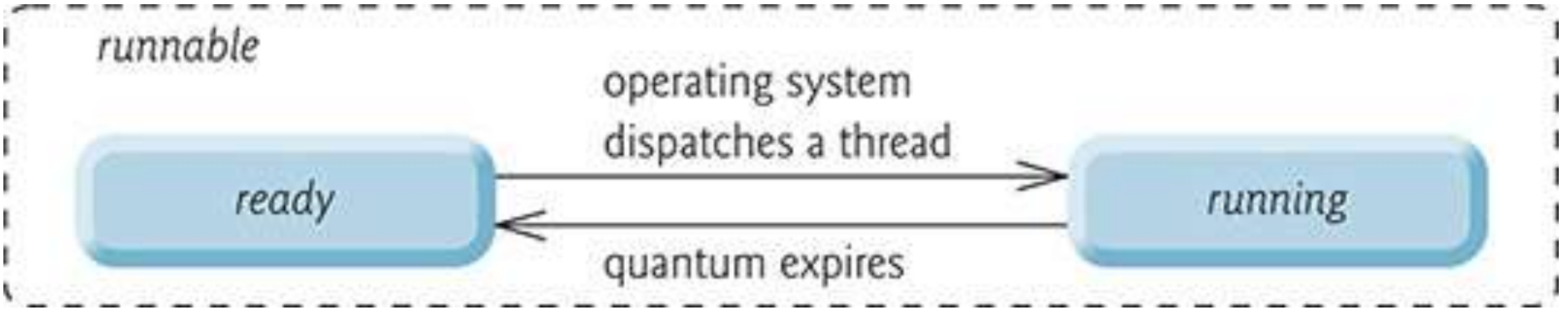
A *runnable* thread transitions to the ***blocked*** state when it attempts to perform a task that cannot be completed immediately and it must temporarily wait until that task completes. For example, when a thread issues an input/output request, the operating system blocks the thread from executing until that I/O request completes—at that point, the *blocked* thread transitions to the *runnable* state, so it can resume execution. A *blocked* thread cannot use a processor, even if one is available.

Terminated State

A *runnable* thread enters the ***terminated*** state (sometimes called the ***dead*** state) when it successfully completes its task or otherwise terminates (perhaps due to an error). In the UML state diagram of Fig. on first slide , the *terminated* state is followed by the UML final state (the bull's-eye symbol) to indicate the endof the state transitions.

Operating-System View of the *Runnable* State

At the operating system level, Java's *runnable* state typically encompasses *two separate* states (Fig. first slide). The operating system hides these states from the JVM, which sees only the *runnable* state. When a thread first transitions to the *runnable* state from the *new* state, it's in the ***ready*** state. A *ready* thread enters the ***running*** state (i.e., begins executing) when the operating system assigns it to a processor—also known as **dispatching the thread**. In most operating systems, each thread is given a small amount of processor time—called a **quantum** or **timeslice**—with which to perform its task. Deciding how large the quantum should be is a key topic in operating systems courses. When its quantum expires, the thread returns to the *ready* state, and the operating system assigns another thread to the processor. Transitions between the *ready* and *running* states are handled solely by the operating system. The JVM does not "see" the transitions—it simply views the thread as being *runnable* and leaves it up to the operating system to transition the thread between *ready* and *running*. The process that an operating system uses to determine which thread to dispatch is called **thread scheduling** and is dependent on thread priorities.


runnable
operating system
dispatches a thread
ready
running
quantum expires

Thread Priorities and Thread Scheduling

Every Java thread has a **thread priority** that helps determine the order in which threads are scheduled. Each new thread inherits the priority of the thread that created it. Informally, higher-priority threads are more important to a program and should be allocated processor time before lower-priority threads. *Nevertheless, thread priorities cannot guarantee the order in which threads execute.*

It's recommended that you do not explicitly create and use* Thread*s to implement concurrency, but rather use the* Executor *interface.

The Thread class does contain some useful static methods, which you *will* use later in the chapter.

Most operating systems support timeslicing, which enables threads of equal priority to share a processor. Without timeslicing, each thread in a set of equal-priority threads runs to completion (unless it leaves the *runnable* state and enters the *waiting* or *timed waiting* state, or gets interrupted by a higher-priority thread) before other threads of equal priority get a chance to execute. With timeslicing, even if a thread has *not* finished executing when its quantum expires, the processor is taken away from the thread and given to the next thread of equal priority, if one is available.

- An *operating system's* **thread scheduler** determines which thread runs next. One simple thread-scheduler implementation keeps the highest-priority thread *running* at all times and, if there's more than one highest-priority thread, ensures that all such threads execute for a quantum each in **round-robin** fashion. This process continues until all threads run to completion.

Creating and Executing Threads with the Executor Framework

- ***Creating Concurrent Tasks with the Runnable Interface***

You implement the **Runnable** interface (of package java.lang) to specify a task that can execute concurrently with other tasks. The Runnable interface declares the single method **run**, which contains the code that defines the task that a Runnable object should perform.

Executing Runnable Objects with an Executor

To allow a Runnable to perform its task, you must execute it. An Executor object executes Runnables. It does this by creating and managing a group of threads called a thread pool. When an Executor begins executing a Runnable, the Executor calls the Runnable object's run method, which executes in the new thread. The Executor interface declares a single method named execute which accepts a Runnable as an argument. The Executor assigns every Runnable passed to its execute method to one of the available threads in the thread pool. If there are no available threads, the Executor creates a new thread or waits for a thread to become available and assigns that thread the Runnable that was passed to method execute.

Executing Runnable Objects with an Executor

To allow a Runnable to perform its task, you must execute it. An Executor object executes Runnables. It does this by creating and managing a group of threads called a thread pool. When an Executor begins executing a Runnable, the Executor calls the Runnable object's run method, which executes in the new thread.

The Executor interface declares a single method named execute which accepts a Runnable as an argument. The Executor assigns every Runnable passed to its execute method to one of the available threads in the thread pool. If there are no available threads, the Executor creates a new thread or waits for a thread to become available and assigns that thread the Runnable that was passed to method execute.

```
public static void main(String[] args)
{
 // create and name each runnable
PrintTask task1 = new PrintTask("task1");
PrintTask task2 = new PrintTask("task2");
PrintTask task3 = new PrintTask("task3");

System.out.println("Starting Executor");

 // create ExecutorService to manage threads
ExecutorService executorService =
Executors.newCachedThreadPool();

 // start the three PrintTasks
executorService.execute(task1); // start task1
executorService.execute(task2); // start task2
executorService.execute(task3); // start task3

// shut down ExecutorService--it decides when to shut down
threads
executorService.shutdown();
```

Advantages of using an Executor

Using an Executor has many advantages over creating threads yourself. Executors can *reuse existing threads* to eliminate the overhead of creating a new thread for each task and can improve performance by *optimizing the number of threads* to ensure that the processor stays busy, without creating so many threads that the application runs out of resources.

Thread Synchronization

There are two types of thread synchronization

- mutual exclusive and
- inter-thread communication.

Mutual Exclusive

- Synchronized method.
- Synchronized block.
- Static synchronization.
- Cooperation (Inter-thread communication in java)

Thread Synchronization

When multiple threads share an object and it's *modified* by one or more of them, indeterminate results may occur unless access to the shared object is managed properly.

The problem can be solved by giving only one thread at a time *exclusive access* to code that accesses the shared object.

During that time, other threads desiring to access the object are kept waiting. When the thread with exclusive access finishes accessing the object, one of the waiting threads is allowed to proceed.

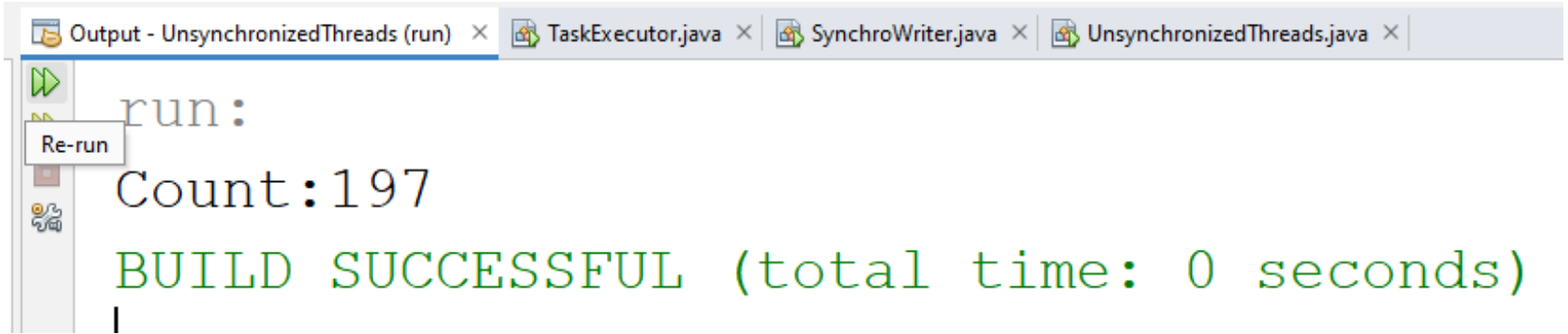
This process, called **thread synchronization**, coordinates access to shared data by multiple concurrent threads. By synchronizing threads in this manner, you can ensure that each thread accessing a shared object *excludes* all other threads from doing so simultaneously—this is called **mutual exclusion**.

Mutable & Immutable Data

Actually, thread synchronization is necessary *only* for shared **mutable data**, i.e., data that may *change* during its lifetime. With shared **immutable data** that will *not* change, it's not possible for a thread to see old or incorrect values as a result of another thread's manipulation of that data.

When you share *immutable data* across threads, declare the corresponding data fields final to indicate that the values of the variables will *not* change after they're initialized. This prevents accidental modification of the shared data, which could compromise thread safety. *Labeling object references as final indicates that the reference will not change, but it does not guarantee that the referenced object is immutable—this depends entirely on the object's properties.* However, it's still good practice to mark references that will not change as final.

```
class Adder implements Runnable {
    @Override
     public void run() {
         try{
             for(int i=0; i< 100; i++){
                 UnsynchronizedThreads.count[0]+=1;
                 Thread.sleep(1);
             }
         }catch(Exception e){}
    }
}
```



Solution:

```
class Adder implements Runnable {
    @Override
    public void run() {
        try{
            for(int i=0; i< 100; i++)
            synchronized (UnsynchronizedThreads.count){
                UnsynchronizedThreads.count[0]+=1;
                Thread.sleep(1);
            }
        }catch(Exception e){}
    }
}
```

Monitors

A common way to perform synchronization is to use Java's built-in **monitors**. Every object has a monitor and a **monitor lock** (or **intrinsic lock**). The monitor ensures that its object's monitor lock is held by a maximum of only one thread at any time.

Monitors and monitor locks can thus be used to enforce mutual exclusion.

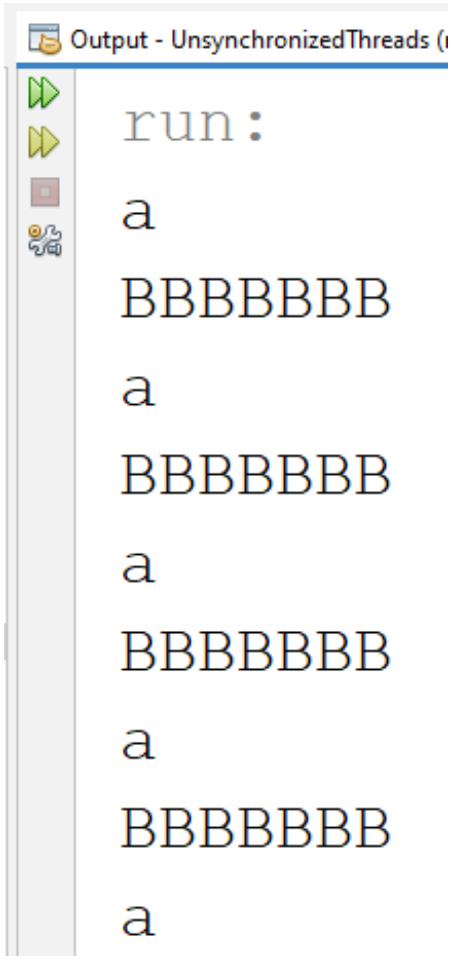
If an operation requires the executing thread to *hold a lock* while the operation is performed, a thread must *acquire the lock* before proceeding with the operation. Other threads attempting to perform an operation that requires the same lock will be *blocked* until the first thread *releases the lock*, at which point the *blocked* threads may attempt to acquire the lock and proceed with the operation.

- To specify that a thread must hold a monitor lock to execute a block of code, the code should be placed in a synchronized statement. Such code is said to be guarded by the monitor lock; a thread must acquire the lock to execute the guarded statements. The monitor allows only one thread at a time to execute statements within synchronized statements that lock on the same object, as only one thread at a time can hold the monitor lock.

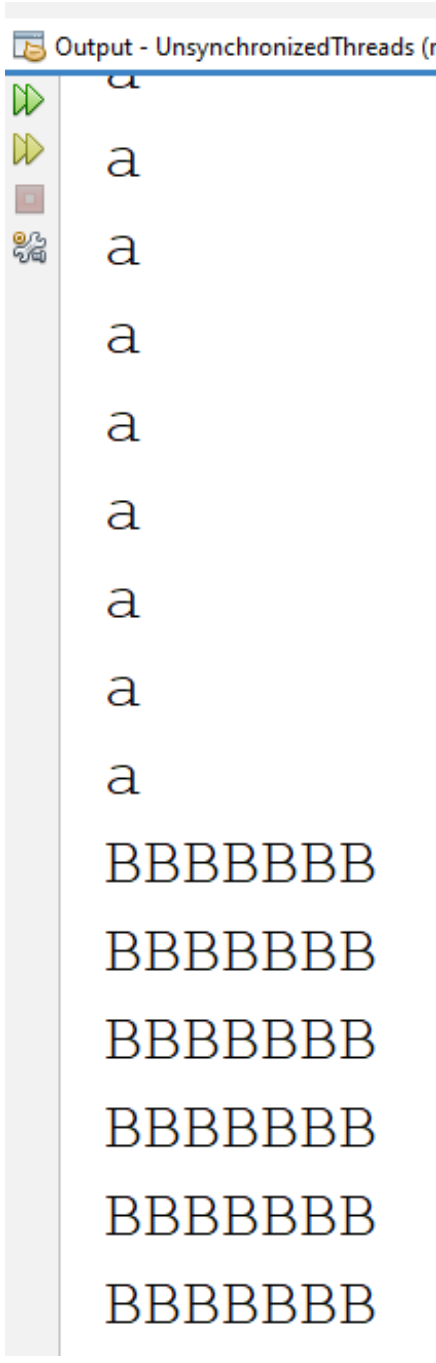
  The synchronized statements are declared using the synchronized keyword: (Example)

Method synchronization

```
class Printer{
    void printText(String txt){

            for(int i=0; i< 100; i++)
            {
                System.out.println(txt);
                try{
                    Thread.sleep(1);
                }catch(Exception e){}
            }

    }
}
```

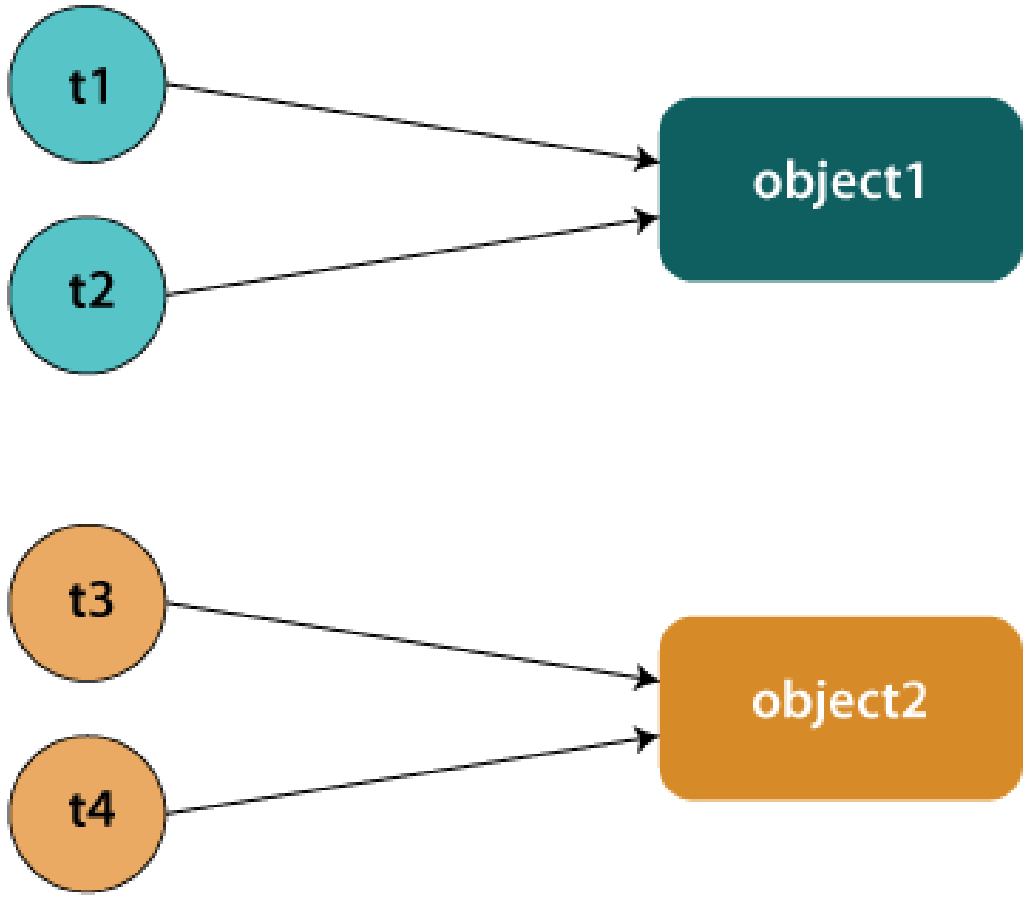


```
class Printer{
    synchronized void printText(String txt){

            for(int i=0; i< 100; i++)
            {
                System.out.println(txt);
                try{
                    Thread.sleep(1);
                }catch(Exception e){}
            }

    }
}
```



Static Synchronization

If you make any static method as synchronized, the lock will be on the class not on object.



Problem without static synchronization

Suppose there are two objects of a shared class (e.g. Table) named object1 and object2. In case of synchronized method and synchronized block there cannot be interference between t1 and t2 or t3 and t4 because t1 and t2 both refers to a common object that have a single lock. But there can be interference between t1 and t3 or t2 and t4 because t1 acquires another lock and t3 acquires another lock. We don't want interference between t1 and t3 or t2 and t4. Static synchronization solves this problem.

```
class Printer{
    synchronized static void printText(String txt){

            for(int i=0; i< 10; i++)
            {
                System.out.println(txt);
                try{
                    Thread.sleep(400);
                }catch(Exception e){}
            }

    }
}
```